

Appendix

A Details of Markov Decision Process-based Reasoning Agent

To systematically capture the sequential dependencies inherent in SQL rewriting, we cast the process as a Markov Decision Process (MDP) $(\mathcal{S}, \mathcal{A}, T, r, \gamma)$. In this formulation:

States. Each $S_t \in \mathcal{S}$ encodes the current SQL query after t rewrite steps, including any injected hints or syntax repairs. The initial state is the user’s original query:

$$S_0 = Q_{\text{orig}}.$$

Actions. The action space $\mathcal{A}$ comprises

- *Refinement actions* R_t : targeted rewrites such as join reordering, predicate pushdown, common-table-expression (CTE) extraction, or hint injection.
- A *terminal action* A : emit the final rewritten query and proceed to verification.

Transitions. Applying R_t to S_t yields

$$S_{t+1} = T(S_t, R_t) = S_t + R_t,$$

where the “+” operator denotes syntactic transformation of the SQL AST. Choosing A halts further rewrites:

$$S_{t+1} = T(S_t, A) = S_t + A.$$

The resulting deterministic transition is:

$$S_{t+1} = \begin{cases} T(S_t, R_t) = S_t + R_t, & \text{if a refinement is applied,} \\ T(S_t, A) = S_t + A, & \text{if rewriting terminates.} \end{cases}$$

Reward. To guide the search toward efficient rewrites, we define a domain-specific *reward function*:

$$r(S_t, R_t) = \text{Cost}(S_t) - \text{Cost}(S_{t+1}),$$

where $\text{Cost}(\cdot)$ is the reasoning LLM estimated execution cost (via EXPLAIN). Positive reward indicates an estimated performance gain.

Policy. The agent’s behavior is governed by a policy

$$\pi_\theta(a | S_t) = \text{LLM}_\theta(a | \text{prompt}(S_t)),$$

implemented by a reasoning-specialized LLM (Deepseek-R1). At each step, the LLM is prompted with the current query state, schema statistics, and recent rewrite history, and produces a probability distribution over $\mathcal{A}$.

Objective. Under discount factor $\gamma \in (0, 1]$, we seek the policy that maximizes expected cumulative reward:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi} \left[\sum_{k=0}^{T-1} \gamma^k r(S_k, a_k) \right].$$

Equivalently, the optimal Q-function satisfies the Bellman equation:

$$Q^*(S_t, a_t) = r(S_t, a_t) + \gamma \max_{a'} Q^*(S_{t+1}, a').$$

Rationale. By framing query rewriting as an MDP, we obtain:

- (1) *Cumulative optimization*: the agent evaluates rewrites on their long-term impact, avoiding local gains that hinder subsequent optimizations.

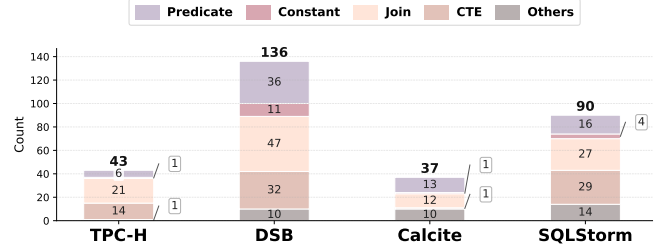


Figure 1: Rewrite Categories for Different Benchmarks

- (2) *Explicit dependency tracking*: each S_t retains full query context, ensuring that later actions (e.g., predicate pushdowns) respect earlier structural changes.
- (3) *LLM-aligned planning*: Deepseek-R1’s chain-of-thought generation naturally corresponds to sampling from π_θ over multiple timesteps, allowing it to propose coherent rewrite trajectories rather than isolated edits.

By treating each rewrite as a state transition, we make explicit how one transformation *enables* or *rules out* subsequent optimizations. To guide the search toward efficient and semantically sound plans, we define a hybrid *reward function*

$$r(S_t, R_t) = \alpha(\text{Cost}(S_t) - \text{Cost}(S_{t+1})) + (1 - \alpha) \text{Score}(S_t, R_t),$$

where:

- $\text{Cost}(\cdot)$ is the optimizer’s estimated execution cost (via EXPLAIN).
- $\text{Score}(S_t, R_t) \in [0, 1]$ is a semantic-and-plan coherence score produced by the reasoning LLM.

Here $\alpha \in [0, 1]$ balances between purely cost-driven improvements and the LLM’s holistic judgment. Intuitively, this hybrid reward leverages the optimizer to quantify immediate cost gains while relying on the LLM to correct for estimation biases and ensure semantic fidelity.

B Details of StackOverflow Benchmarks

Query Generation. SQLStorm [4] generates SQL queries of varying complexity (P1–P7) for benchmarking. We randomly sampled 20 queries per tier from P1–P5 to generate 100 SQLs, as P6–P7 yielded low generation success rates [4]. Each query is executed to filter out syntactic errors. Since LLM-generated queries adopt general dialects that cannot be directly parsed or rewritten by rule-based Calcite engines in [3, 5, 8] (e.g., INTERVAL ‘1’ YEAR vs. INTERVAL 1 YEAR, or LISTAGG vs. STRING_AGG), we manually normalized dialect inconsistencies to ensure compatibility across Calcite and PostgreSQL, discarding unresolvable cases and retaining 43 valid queries.

Rewrite Performance. As shown in Table 1, outperforms all baselines on the StackOverflow-Math benchmark. Specifically, achieves a 73.3% reduction in execution time compared to the original queries, 72.3% compared to LR, 70.5% compared to LLM-R², 72.4% compared to R-Bot, and 52.7% compared to LLM Agent. This highlights the

Table 1: Different Methods on SQLStorm Benchmark

Methods	SQLStorm (SF=10)			
	Mean	Median	75th	95th
Original	48.02	9.47	20.80	300.00
LearnedRewrite	46.25	9.43	18.35	300.00
LLM-R ² (GPT-4o)	47.08	9.19	16.06	300.00
LLM-R ² (Claude-3.7)	46.97	8.40	16.30	300.00
LLM-R ² (DS-R1)	43.42	9.21	14.14	300.00
R-Bot (GPT-4o)	46.43	9.37	16.02	300.00
R-Bot (Claude-3.7)	52.06	9.52	19.25	300.00
R-Bot (DS-R1)	50.57	9.46	17.39	300.00
LLM Agent (GPT-4o)	50.90	7.96	26.73	300.00
LLM Agent (DS-R1)	27.14	6.38	9.40	272.55
LLM Agent (DS-V3)	54.92	10.33	23.60	300.00
LLM Agent (Claude-3.7)	36.23	7.58	21.86	283.42
◦ (DS-R1 + Claude-3.7)	12.83	5.16	9.05	16.11
★ (DS-R1 + Claude-3.7)	12.61	5.03	7.96	15.89

generalization capability of our framework beyond standard benchmarks. In addition, rule-based rewriters [3, 5, 8] often fail to generate valid rewrites even after manual dialect normalization, as they rely heavily on specific rewrite engines such as Apache Calcite [1], which enforce strict SQL dialect constraints. In contrast, QUITE exhibits greater robustness to dialect variation, overcoming the rule-pattern limitations.

In our evaluation, even after dialect normalization, LLM-R² and R-Bot still failed to output valid rewrites in many cases, due to their brittle keyword matching/replace. We implemented minimal patches to improve these regex mappings and will commit them upstream after the review. Moreover, SQLStorm’s queries proved less challenging. Don’t say it’s less challenging, which doesn’t sound good for LLM-based rewriting than classic benchmarks, whereas rule-based pipelines [3, 5, 8] remained constrained by engine and pattern coverage.

C Rewrite Details Analysis

Rewrite Types Analysis. We report successful rewrites that achieve over a 10% performance improvement and maintain equivalence across four benchmarks in our experiments. As shown in Figure 1, the main categories are Join (e.g., join order optimization, use EXISTS instead of complex JOIN) for 29.97%, CTE (e.g., using CTE to avoid repeated and redundant calculations) for 21.29%, and Predicate (e.g., predicate pushdown) for 19.88%, together account for 71.14% of all successful rewrites. The remaining 28.86% correspond to a variety of less frequent, task-specific rewrite strategies that complement the dominant three categories. Compared with rule-based baselines [3, 5, 8], performs deeper and more diverse rewrites rather than pattern-matching surface edits.

We count the number of effective rewrites (speedup >10x) across five categories: CTE (25/34), Join (23/34), Predicate (15/34), Others (7/34), and Constant (5/34). We further classify each query by its applied optimization strategies and compute the average speedup to mitigate bias from mixed techniques. “CTE for Nested Subquery

Rewrite” achieves an average speedup of 79.5x, “Join or Filter Push-down into CTE” yields 66.8x, and “join order optimization” provides 39.6x. These results confirm that eliminating redundant subqueries delivers the most substantial performance gains overall, eliminating redundant scans and enables more efficient join reordering and parallel execution [2, 6].

We examine types of query structures that optimizes most effectively. Queries with heavy aggregation, nesting, or late filtering show 10x–105x speedups after rewriting. In DSB and StackOverflow-Math, QUITE decomposes large CTEs and removes redundant calculations, whereas rule-based systems [3, 5, 8] cannot express such CTE transformations. For Calcite queries, removes redundant subqueries and simplifies GROUP BY/UNION, while retaining query equivalence. Note that although some strategies are also available for rule-based rewriting, they usually cannot be applied to real-world queries due to the limitations of pattern matching in rule-based approaches. In contrast, flexibly rewrites queries using LLMs, allowing it to generalize across diverse query patterns. More detailed analyses and examples are provided in [7].

Q&A Units Analysis. We analyzed the retrieval frequencies of our 241 Q&A units on the DSB benchmark. Units from the “CTE” and “Join” categories were retrieved most frequently, at 197 and 191 times, respectively. The most effective knowledge unit was on de-correlation (rewriting correlated subqueries to CTE), which was retrieved 31 times and successfully optimized 6 queries for substantial performance gains. While these categories often yield large performance gains, our agent does not prioritize retrieval by potential impact. Rewrite effectiveness is highly context-specific because a strategy’s benefit is not guaranteed and can even cause performance degradation.

Query Hint Analysis. We analyze the frequency and impact of the most beneficial hints. A reduction in execution time of at least 10% is considered an improvement, while changes within $\pm 10\%$ are regarded as comparable performance. For join strategies, the NoNestLoop hint improved in 10 out of 25 cases and achieved comparable performance in 13 cases. The HashJoin hint improved 9 out of 19 cases and achieved comparable performance in another 9. For common table expressions (CTEs), the NOT_MATERIALIZE hint was selectively applied in 6 cases, improving 4 and maintaining performance in 2. In cardinality correction, the Rows hint was the most frequently applied (27 total cases), indicating that estimation errors are a major source of performance degradation. This hint contributed to most of the observed gains, with 18 cases showing improvement and 6 remaining stable.

D Rewrite Examples Beyond Rules

Existing rule sets are inherently limited and can be suboptimal. In contrast, our framework leverages the agents to explore and discover more effective rewrite strategies—delivering performance gains that traditional rule-based methods cannot match. Below we present representative examples:

P1: Normalizes arithmetic predicates and converts redundant GROUP BY to ‘SELECT DISTINCT’.

By inverting the join expressions (DEPTNO - 10 = SAL + 1) into a simpler filter (SAL = DEPTNO - 11), then replacing the grouped

projection of SAL + 1 per DEPTNO with a SELECT DISTINCT DEPTNO on the simplified predicate, thereby eliminating both the derived-table grouping and the extra computed columns.

Original Query (34.91s):

```
SELECT t4.NAME
FROM (
  SELECT NAME,
         DEPTNO,
         DEPTNO - 10 AS DEPTNOMINUS
  FROM DEPT
) AS t4
INNER JOIN (
  SELECT DEPTNO,
         SAL + 1 AS f9
  FROM EMP
  GROUP BY DEPTNO, SAL + 1
) AS t6
ON t4.DEPTNOMINUS = t6.f9
AND t4.DEPTNO = t6.DEPTNO;
```

Rewritten Query (0.74s):

```
SELECT t4.NAME
FROM (
  SELECT NAME,
         DEPTNO,
         DEPTNO - 10 AS DEPTNOMINUS
  FROM DEPT
) AS t4
INNER JOIN (
  SELECT DEPTNO,
         SAL + 1 AS f9
  FROM EMP
  GROUP BY DEPTNO, SAL + 1
) AS t6
ON t4.DEPTNOMINUS = t6.f9
AND t4.DEPTNO = t6.DEPTNO;
```

P2: Applies subquery flattening and aggregation elimination

Removing the two grouped derived tables and replacing them with a single 'EXISTS' predicate plus 'DISTINCT', thereby collapsing the LEFT JOIN+GROUP BY into a simple filter on the base table.

Original Query (38.01s):

```
SELECT t6.ENAME
FROM (
  SELECT JOB
  FROM EMP
  WHERE ENAME = 'A'
  GROUP BY JOB
) AS t5
LEFT JOIN (
  SELECT ENAME, JOB
  FROM EMP
  GROUP BY ENAME, JOB
) AS t6
```

```
ON t5.JOB = t6.JOB
GROUP BY t6.ENAME;
```

Rewritten Query (1.86s):

```
SELECT DISTINCT E.ENAME
FROM EMP E
WHERE EXISTS (
  SELECT 1
  FROM EMP A
  WHERE A.ENAME = 'A'
        AND A.JOB = E.JOB
);
```

P3: Eliminates subqueries and joins via tautology removal

Recognizes that both derived tables filter on the same predicates ('DEPTNO = 20 AND SAL > 1000') and that the LEFT JOIN+GROUP BY merely re-projects SAL values, so it collapses the logic into a single scan with the original filters and removes all unnecessary grouping and joins.

Original Query (1.99s):

```
SELECT t0.SAL
FROM (
  SELECT EMPNO, ENAME, JOB, MGR, HIREDATE, SAL, COMM,
         SLACKER, SAL AS SAL0, SAL = 4 AS f9
  FROM EMPNULLABLES
  WHERE DEPTNO = 20 AND SAL > 1000
) AS t0
INNER JOIN (
  SELECT t2.SAL, t5.SAL AS SAL0, t5.f9
  FROM (
    SELECT EMPNO, ENAME, JOB, MGR, HIREDATE,
           SAL, COMM, SLACKER
    FROM EMPNULLABLES
    WHERE DEPTNO = 20 AND SAL > 1000
  ) AS t2
  INNER JOIN (
    SELECT SAL, SAL = 4 AS f9
    FROM EMPNULLABLES
    WHERE DEPTNO = 20 AND SAL > 1000
    GROUP BY SAL, SAL = 4
  ) AS t5
  ON t2.SAL = t5.SAL OR t5.f9
  GROUP BY t2.SAL, t5.SAL, t5.f9
  HAVING t5.SAL = t2.SAL
) AS t8
ON t0.SAL = t8.SAL0 AND t0.f9 = t8.f9;
```

Rewritten Query (1.11s):

```
SELECT SAL
FROM EMPNULLABLES
WHERE DEPTNO = 20
AND SAL > 1000;
```

P4: Applies nested-subquery flattening and redundant

Join elimination—recognizes that all derived tables apply the same predicates ('DEPTNO = 20 AND SAL > 1000') and simply project 'SAL', so it collapses the nested SELECTs and INNER JOIN+GROUP BY into a single filtered scan on 'EMPNULLABLES'.

Original Query (2.07s):

```
SELECT t12.SAL
FROM (
  SELECT *
  FROM (
    SELECT EMPNO, ENAME, JOB, MGR, HIREDATE,
           SAL, COMM, SLACKER,
           SAL AS SAL0, SAL = 4 AS f9
    FROM EMPNULLABLES
    WHERE DEPTNO = 20 AND SAL > 1000
  ) AS t11
  WHERE SAL > 1000
) AS t12
INNER JOIN (
  SELECT t14.SAL, t17.SAL AS SAL0, t17.f9
  FROM (
    SELECT EMPNO, ENAME, JOB, MGR, HIREDATE,
           SAL, COMM, SLACKER
    FROM EMPNULLABLES
    WHERE DEPTNO = 20 AND SAL > 1000
  ) AS t14
  INNER JOIN (
    SELECT SAL, SAL = 4 AS f9
    FROM EMPNULLABLES
    WHERE DEPTNO = 20 AND SAL > 1000
    GROUP BY SAL, SAL = 4
  ) AS t17
  ON t14.SAL = t17.SAL OR t17.f9
  GROUP BY t14.SAL, t17.SAL, t17.f9
  HAVING t17.SAL = t14.SAL
) AS t20
  ON t12.SAL = t20.SAL0
 AND t12.f9 = t20.f9;
```

Rewritten Query (1.10s):

```
SELECT SAL
FROM EMPNULLABLES
WHERE DEPTNO = 20
      AND SAL > 1000;
```

P5: Applies join-elimination and aggregation unification

Recognizing that both derived tables group by the same key 'SAL' and then inner-join on 'SAL' merely re-produces the same aggregation, so it collapses the two into one single GROUP BY, removing the redundant join.

Original Query (3.46s):

```
SELECT SUM(CAST(t3.f1 * t4.SAL AS INTEGER))
FROM (
  SELECT SAL, COUNT(*) AS f1
```

```
FROM EMP
  GROUP BY SAL
) AS t3
INNER JOIN (
  SELECT SAL
  FROM EMP
  GROUP BY SAL
) AS t4
  ON t3.SAL = t4.SAL;
```

Rewritten Query (1.80s):

```
SELECT SUM(CAST(f1 * SAL AS INTEGER))
FROM (
  SELECT SAL, COUNT(*) AS f1
  FROM EMP
  GROUP BY SAL
) AS t;
```

P6: Applies subquery flattening and EXISTS-to-IN unnesting

Eliminating the nested derived tables and GROUP BY on a constant expression and converting the correlated EXISTS into a simple IN predicate at the top level.

Original Query (0.29s):

```
SELECT NAME
FROM (
  SELECT DEPTNO, NAME
  FROM DEPT
  WHERE EXISTS (
    SELECT 1
    FROM (
      SELECT 2 * DEPTNO AS f0
      FROM DEPT
      GROUP BY 2 * DEPTNO
    ) AS t5
    WHERE DEPTNO = t5.f0
  )
) AS t2;
```

Rewritten Query (0.13s):

```
SELECT NAME
FROM DEPT
WHERE DEPTNO IN (
  SELECT 2 * DEPTNO
  FROM DEPT
);
```

P7: Applies limit-pushdown through subquery reordering

By moving the FETCH FIRST 10 ROWS ONLY inside the EMP scan, the optimizer retrieves only the top-10 highest-salary employees before performing the join, rather than joining the entire EMP and DEPT tables and then limiting.

Original Query (5.56s):

```
SELECT *
```

```

FROM EMP
LEFT JOIN (SELECT * FROM DEPT) AS t
  ON EMP.DEPTNO = t.DEPTNO
ORDER BY EMP.SAL
FETCH NEXT 10 ROWS ONLY;

```

Rewritten Query (1.12s):

```

SELECT e.*, t.*
FROM (
  SELECT *
  FROM EMP
  ORDER BY SAL
  FETCH FIRST 10 ROWS ONLY
) e
LEFT JOIN DEPT t
  ON e.DEPTNO = t.DEPTNO
ORDER BY e.SAL;

```

P8: Normalizes quantified predicates by converting '> SOME (subquery)' into a simple comparison against 'MIN(...)'

Rewriting $x > \text{SOME}(\text{subquery})$ as $x > (\text{SELECT MIN}(\dots) \text{ FROM subquery})$, which eliminates the quantifier and lets the optimizer treat it as a simple filter.

Original Query (103.86s):

```

SELECT *
FROM emp e1
WHERE e1.empno > SOME (
  SELECT deptno
  FROM dept
);

```

Rewritten Query (19.44s):

```

SELECT *
FROM emp e1
WHERE e1.empno > (
  SELECT MIN(deptno)
  FROM dept
);

```

P9: Join-elimination and DISTINCT projection

Recognizes that a full outer join grouping on identical columns adds no new information beyond the distinct values themselves, so it drops the join and grouping entirely, replacing it with a single `SELECT DISTINCT mgr` over the base table.

Original Query (>300s):

```

SELECT
  e.mgr,
  d.mgr
FROM
  emp AS e
  FULL OUTER JOIN emp AS d
    ON e.mgr = d.mgr
GROUP BY

```

```

  e.mgr,
  d.mgr;

```

Rewritten Query (10.74s):

```

SELECT DISTINCT mgr, mgr
FROM emp;

```

P10: Applies multi-CTE filter materialization and decorrelation of EXISTS subqueries

First isolating qualifying orders and lineitems into CTEs ('filtered_orders', 'lineitem_filtered') so that the status and date filters are applied once, and then referencing these CTEs in the main query and in the 'EXISTS'/'NOT EXISTS' checks. This lifts correlated predicates out of the main scan, avoids repeated evaluation, and lets the optimizer plan each CTE independently.

Original Query (>300s):

```

SELECT
  s.s_name,
  COUNT(*) AS numwait
FROM
  supplier AS s
  JOIN nation AS n
    ON s.s_nationkey = n.n_nationkey
  JOIN lineitem AS l1
    ON s.s_suppkey = l1.l_suppkey
  JOIN orders AS o
    ON l1.l_orderkey = o.o_orderkey
WHERE
  o.o_orderstatus = 'F'
  AND l1.l_receiptdate > l1.l_commitdate
  AND EXISTS (
    SELECT 1
    FROM lineitem AS l2
    WHERE l2.l_orderkey = l1.l_orderkey
      AND l2.l_suppkey <> l1.l_suppkey
  )
  AND NOT EXISTS (
    SELECT 1
    FROM lineitem AS l3
    WHERE l3.l_orderkey = l1.l_orderkey
      AND l3.l_suppkey <> l1.l_suppkey
      AND l3.l_receiptdate > l3.l_commitdate
  )
  AND n.n_name = 'SAUDI ARABIA'
GROUP BY
  s.s_name
ORDER BY
  numwait DESC,
  s.s_name
LIMIT 100;

```

Rewritten Query (86.50s):

```

WITH filtered_orders AS (

```

```

SELECT o_orderkey
FROM orders
WHERE o_orderstatus = 'F'
),
lineitem_filtered AS (
SELECT
    l_orderkey,
    l_suppkey,
    l_receiptdate,
    l_commitdate
FROM lineitem
WHERE l_orderkey IN (
SELECT o_orderkey FROM filtered_orders)
),
filtered_suppliers AS (
SELECT
    s_suppkey,
    s_name
FROM supplier
JOIN nation ON s_nationkey = n_nationkey
WHERE n_name = 'SAUDI ARABIA'
)
SELECT
    s.s_name,
    COUNT(*) AS numwait
FROM
    filtered_suppliers s
JOIN lineitem_filtered l1
    ON s.s_suppkey = l1.l_suppkey
WHERE
    l1.l_receiptdate > l1.l_commitdate
    AND EXISTS (
        SELECT 1
        FROM lineitem_filtered l2
        WHERE
            l2.l_orderkey = l1.l_orderkey
            AND l2.l_suppkey <> l1.l_suppkey
    )
    AND NOT EXISTS (
        SELECT 1
        FROM lineitem_filtered l3
        WHERE
            l3.l_orderkey = l1.l_orderkey
            AND l3.l_suppkey <> l1.l_suppkey
            AND l3.l_receiptdate > l3.l_commitdate
    )
GROUP BY
    s.s_name
ORDER BY
    numwait DESC,
    s.s_name
LIMIT 100;

```

processing over heterogeneous data sources. In *Proceedings of the 2018 International Conference on Management of Data*. 221–230.

- [2] Nicolas Bruno, César Galindo-Legaria, and Milind Joshi. 2025. Query Decorrelation in the Fabric Data Warehouse. In *Companion of the 2025 International Conference on Management of Data*. 297–309.
- [3] Zhaodonghui Li, Haitao Yuan, Huiming Wang, Gao Cong, and Lidong Bing. 2024. LLM-R2: A Large Language Model Enhanced Rule-based Rewrite System for Boosting Query Efficiency. *arXiv preprint arXiv:2404.12872* (2024).
- [4] Tobias Schmidt, Viktor Leis, Peter Boncz, and Thomas Neumann. 2025. SQL-Storm: Taking Database Benchmarking into the LLM Era. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4144–4157.
- [5] Zhaoyan Sun, Xuanhe Zhou, and Guoliang Li. 2025. R-Bot: An LLM-based Query Rewrite System. *Proceedings of the VLDB Endowment* 18, 12 (2025), 5031 – 5044.
- [6] TiDB. 2025. Decorrelation of Correlated Subquery . <https://docs.pingcap.com/tidb/stable/correlated-subquery-optimization/> Accessed: 2025-10-19.
- [7] Yuyang Song. 2025. Github Repository of QUITTE Rewrite types. https://github.com/Yuyang-Song/QUITTE/blob/main/documents/rewrite_types.md
- [8] Xuanhe Zhou, Guoliang Li, Chengliang Chai, and Jianhua Feng. 2021. A learned query rewrite system using monte carlo tree search. *Proceedings of the VLDB Endowment* 15, 1 (2021), 46–58.

References

- [1] Edmon Begoli, Jesús Camacho-Rodríguez, Julian Hyde, Michael J Mior, and Daniel Lemire. 2018. Apache calcite: A foundational framework for optimized query